ASSIGNMENT 8

Optimize Anything

Technical Report

Discover a process. Formulate the problem.
Apply optimization. Prove the improvement.

Ankita Sagar

Roll No: 241071063

Optimization Techniques

Tracks A-J | AI-assisted development | Reproducible experiments

6 November 2026

Contents

Abstract 4

1. Experimental method 4

1.1 Common workflow . . . 4
1.2 Environment . . . 4
1.3 Measurement policy . . . 5

2. Results overview 5

3. Track A - Continuous optimization 5

3.1 Formulation . . . 5
3.2 Methods and results . . . 5
3.3 Correctness and limitations . . . 6

4. Track B - Discrete optimization 6

4.1 Knapsack formulation . . . 6
4.2 QUBO and Ising model . . . 6
4.3 Correctness and limitations . . . 7

5. Track C - Quantum optimization 7

5.1 QAOA on Track B's QUBO . . . 7
5.2 VQE, quantum optimizers and QNN . . . 7
5.3 Correctness and limitations . . . 7

6. Track D - Submodular, parallel, distributed and multi-task optimization 8

6.1 Submodular maximum coverage . . . 8
6.2 Parallel scheduling . . . 8
6.3 FedAvg . . . 8
6.4 MGDA and GradNorm . . . 8
6.5 Correctness and limitations . . . 8

7. Track E - Main case study: project-workflow scheduling 9

7.1 Process and baseline evidence . . . 9
7.2 Formulation . . . 9
7.3 Strategy selection . . . 9
7.4 Results . . . 9
7.5 Correctness and limitations . . . 10

8. Track F - Compiler optimization 10

8.1 Compiler and passes . . . 10
8.2 Results . . . 10
8.3 Correctness and limitations . . . 10

9. Track G - xv6 operating-system optimization **11**

9.1 Kernel change and objective . . . 11
9.2 Results . . . 11
9.3 Correctness and limitations . . . 11

10. Track H - DBMS optimization **11**

10.1 Formulation . . . 11
10.2 Results . . . 11
10.3 Correctness and limitations . . . 12

11. Track I - Quantum algorithms and QML comparison **12**

11.1 Algorithms . . . 12
11.2 QML versus classical learning . . . 12
11.3 Correctness and limitations . . . 13

12. Track J - Software-engineering optimization **13**

12.1 Process and formulation . . . 13
12.2 Results . . . 13
12.3 Correctness and limitations . . . 13

13. Cross-track method selection **14**

14. AI-assisted development and critical review **14**

15. Answers to the ten final questions **14**

1. What process was chosen, and what evidence showed that it needed optimization? . . . 14
2. How were the objective function and constraints formulated? . . . 14
3. Which Chapter 6 methods were tested, and why were they appropriate? . . . 15
4. How did optimized solutions compare with baselines? . . . 15
5. What trade-offs, failures, or limitations were discovered? . . . 15
6. What evidence demonstrates correctness and reproducibility? . . . 15
7. How could the approach adapt if scale, data, constraints, or hardware changed? . . . 15
8. For compiler, xv6 and DBMS tracks, what was implemented and how was correctness preserved? . . . 15
9. What advantages and limits did quantum and parallel experiments reveal? . . . 15
10. Which AI tools were used, how did their outputs differ, and which errors were fixed? . . . 16

16. Reproducibility map **16**

17. Conclusion **16**

Repository: Sagarankita/OT_LAB08 (public, verified snapshot `9757572`)

Abstract

This work treats optimization as a portfolio of experimentally testable decisions rather than as one large application. Ten compulsory tracks cover continuous, discrete, quantum, submodular, parallel, distributed, compiler, operating-system, database, and software-engineering optimization. The main case study optimizes resource-constrained project scheduling with release times and precedence constraints. Every experiment defines decision variables, an objective, constraints, a baseline, comparison methods, and an independent correctness check.

The results show that the best method depends strongly on variable type, scale, hardware, and the cost of optimization itself. Full-batch gradient descent was best on a small well-conditioned ridge problem, whereas adaptive methods retained larger gaps under the fixed tuning budget. Exact discrete methods solved small knapsack instances but cost more than greedy selection. A critical-path scheduling heuristic improved 24-job makespan by 5.69% in approximately 0.12 ms, while a time-limited MILP was thousands of times slower and could not prove optimality. Compiler passes reduced bytecode size by up to 29.85%; budgeted database indexes reduced mean workload latency by 46.22%; and ILP-based test selection reduced a 671-test suite to 242 tests while preserving measured line coverage and cutting wall time by 22.96%.

The quantum results are deliberately cautious. QAOA, VQE, QNN, QFT, QPE, Grover, compiled Shor order finding, and a gate-model annealing evolution were executed on local simulators. Grover used fewer oracle queries than linear search but was much slower in wall-clock simulation. Quantum and classical classifiers both achieved 100% accuracy on a small matched dataset, while the quantum classifier required thousands of circuit evaluations and substantially more time. These are functional demonstrations, not evidence of practical quantum advantage.

All 68 project tests pass. On 6 November 2026, the environment smoke checks, dependency checks, and all selected per-track independent verifiers were rerun successfully. Eleven genuine AI-generated failures were preserved with their corrections. The cleaned repository was published at `https://github.com/Sagarankita/OT_LAB08`. Remaining submission disclosures are the unavailable exact Chapter 6 class notes, the still-missing genuine second-AI-tool comparison, unavailable full chat exports/screenshots, and unavailable historical per-step commit links because Git was initialized only at final handoff.

1. Experimental method

1.1 Common workflow

Each track follows the same process:

1. Identify a process consuming time, computation, memory, storage, or another resource.
2. Establish a deterministic or repeated baseline under documented conditions.
3. state decision variables, objective, constraints, and success metrics.
4. Select algorithms according to variable type, problem size, guarantees, and hardware.
5. Use AI-assisted implementation, then independently test rather than trusting generated code.
6. Compare methods on the same data, stopping rules, and evaluation conditions.
7. Preserve raw data, seeds, timings, configurations, plots, solver status, and failures.
8. Report negative results and optimization overhead alongside improvements.

1.2 Environment

The final audit used Python 3.11.15 on macOS arm64 with 10 logical CPUs. Important packages were NumPy 2.4.6, SciPy 1.17.1, Matplotlib 3.11.2, PyTorch 2.14.1, PennyLane 0.45.1, pytest 9.1.1, coverage.py 7.17.0, and SQLite 3.53.0. Quantum work used local PennyLane simulators. xv6 used QEMU RISC-V and a pinned official xv6-riscv revision. Detailed environment evidence is in ENVIRONMENT.md.

The audit commands were:

```
.venv/bin/python scripts/check_environment.py
.venv/bin/python -m pip check
.venv/bin/python -m pytest -q
```

The dependency check reported no broken requirements and all 68 tests passed.

1.3 Measurement policy

Stochastic experiments use deterministic seeds and repeated runs where feasible. Setup, tuning, and warm-up are separated from measured execution. Solver time limits and optimality status are retained. A result is called optimal only when independently enumerated or when the solver proves optimality. Simulator output is not described as hardware output, and a query-complexity advantage is not described as a wall-clock speedup.

2. Results overview

Track	Process optimized	Baseline	Main optimized comparison	Principal measured result
A	Continuous ML objectives	Untuned/standard gradient updates	Eight first-order methods plus Newton, Gauss-Newton and curvature methods	GD reached floating-point objective gap in 1.009 ms on the small ridge task
B	Binary selection	Ratio-greedy knapsack	DP, ILP and Branch-and-Bound	Exact methods matched exhaustive optima; greedy mean gap 0.526%
C	Variational quantum optimization	Exact classical diagonalization/enumeration	QAOA, VQE, SPSA, QNG and QNN	QNG VQE error 9.51e-5; noise degraded QAOA mean energy
D	Coverage, parallel execution and learning	Serial/equal/centralized methods	Greedy, process schedules, FedAvg, MGDA and GradNorm	Best fixed-size speedup 1.565x; many fine-grained parallel variants were slower
E	Project workflow scheduling	Fixed-order list scheduler	Critical-path heuristic and MILP	24-job heuristic improved makespan 5.69% in about 0.12 ms
F	Generated toy bytecode	Unoptimized compiler	Four passes and DP pass ordering	Bytecode size reduced 29.85%; outputs remained identical
G	xv6 CPU scheduling	Round robin	Weighted stride scheduling	High-weight CPU job completed 25% sooner; latency target regressed 50%
H	SQLite analytical workload	No added candidate indexes	Greedy and exact DP index selection	Mean workload latency fell 46.22% within 3.5 MiB budget
I	Quantum algorithms and QML	Classical search/models	Direct quantum circuits, VQC and quantum kernel	Query advantage but no wall-clock or QML advantage on simulators
J	CI test execution	Full 671-test suite	Naive, greedy and ILP set cover	ILP retained 2,030 covered lines with 242 tests and 22.96% lower wall time

3. Track A - Continuous optimization

3.1 Formulation

The main benchmark is ridge regression with 128 standardized examples and six continuous parameters:

```
L(w) = (1/(2n)) ||Xw-y||^2 + (lambda/2)||w||^2, lambda=0.05
```

The Hessian eigenvalues are positive (`mu=0.7635`, `L=1.3258`), so the objective is strongly convex with a unique global minimizer. Five seeds control initialization and stochastic order. Every first-order method receives 80 complete data exposures, equal to 10,240 example-gradient evaluations. The tuning phase tests three learning rates per method using a separate seed and is excluded from measured runtime.

3.2 Methods and results

Method	Mean runtime	Mean final objective gap
GD	1.009 ms	0 at saved precision
SGD	29.139 ms	6.48e-6
Mini-batch SGD	2.718 ms	1.26e-5
Momentum	3.013 ms	2.574e-5

Method	Mean runtime	Mean final objective gap
NAG	3.276 ms	2.366e-5
AdaGrad	3.309 ms	1.7916e-4
RMSProp	3.628 ms	3.78071e-3
Adam	4.524 ms	1.23614e-3

GD won on this small deterministic and well-conditioned problem. This does not imply universal dominance: SGD made more Python-level updates, while the adaptive methods had only a narrow tuning budget. Separate examples demonstrate continuous versus discrete optima, convex but not strongly convex behavior, strongly convex convergence, stable and unstable step sizes, non-convex local/global minima, and a saddle point.

Backpropagation was separated from optimization using a manually differentiated 2-3-1 XOR network. Its gradient matched finite differences with relative error `5.02e-10` and matched PyTorch autograd at saved precision. Adam then used the backpropagated gradient to reach 100% training accuracy.

Newton reached a quadratic optimum in one exact solve. Gauss-Newton parameters were within `1.31e-10` of SciPy least squares. An exact Hessian-vector product matched a finite-difference gradient calculation within `3.29e-11`. Expected Fisher curvature matched the logistic Hessian within `4.64e-17`, while diagonal and rank-two approximations had relative errors 0.2127 and 0.6271. The worse low-rank result is retained as a negative result. On a noisy scalar objective, a decaying step reduced final absolute error from `0.0607 +/- 0.0295` to `0.0181 +/- 0.0154`.

3.3 Correctness and limitations

The independent verifier passed 28 checks, including analytical optima, tuning reproduction, equal data budgets, finite-difference gradients, backpropagation, second-order references, Fisher identities, and artifact hashes. The problem is too small to establish rankings for large neural networks, GPUs, sparse data, or ill-conditioned objectives.

Code and evidence: continuous.py, Track A details, and `results/track_a_continuous/20261027T134547.158590Z/`.

4. Track B - Discrete optimization

4.1 Knapsack formulation

For binary item decisions `x_i`, values `v_i`, weights `w_i`, and capacity `C`:

```
maximize sum_i v_i x_i
subject to sum_i w_i x_i <= C
x_i in {0,1}
```

Five seeded 18-item instances were supplied unchanged to ratio greedy, dynamic programming, explicit Branch-and-Bound, SciPy/HiGHS ILP, and exhaustive verification. Each method received the same inputs, one warm-up, and five measured repetitions.

Method	Mean time	Mean optimality gap	Status
Greedy	0.00644 ms	0.5260%	Always feasible; not always optimal
Dynamic programming	0.12360 ms	0%	Matched exhaustive oracle
Branch-and-Bound	0.03367 ms	0%	Matched exhaustive oracle
ILP/HiGHS	2.52765 ms	0%	Matched exhaustive oracle

The exact methods agree on these small cases, but their complexity differs: DP is pseudo-polynomial in capacity, Branch-and-Bound is exponential in the worst case, and ILP has solver overhead.

4.2 QUBO and Ising model

A second problem chooses exactly two of five projects. With reward `S(x)` and cardinality `k=2`:

```
E(x) = -S(x) + P(sum_i x_i-k)^2
```

The sufficient penalty `P=40` exceeds the maximum total reward and forces every global minimizer to be feasible. The optimum is `[1,1,0,0,0]`, score 18, energy -18. A deliberately insufficient penalty `P=1` produces infeasible global minima, demonstrating why penalties require justification. Binary-to-spin conversion uses `x_i=(1-z_i)/2`; QUBO and Ising energies agree for all 32 bitstrings.

4.3 Correctness and limitations

The independent verifier passed 19 checks across 100 timing records, five exhaustive oracles, both penalties, all QUBO/Ising states, feasibility, and the frozen Track C handoff. Results are intentionally small and do not predict large ILP or Branch-and-Bound behavior.

Code and evidence: discrete.py, Track B details, and `results/track_b_discrete/20261027T133954.808190Z/`.

5. Track C - Quantum optimization

5.1 QAOA on Track B's QUBO

The five-qubit cost Hamiltonian preserves Track B's exact coefficients and the convention `x=(1-z)/2`. All 32 computational-basis energies match the classical QUBO and Ising energies with zero maximum error. The classical ground energy is -18.

Depth-three QAOA used three Adam starts for 200 iterations. The best expected energy was -10.3078. Other starts ended at 1.4317 and 2.5355, exposing non-convex initialization sensitivity. Finite-shot samples at 100, 1,000 and 5,000 shots observed the exact feasible optimum in every seed; estimator standard deviation fell from 1.6979 to 0.1331 as shots increased. This reduces measurement variability but does not improve the underlying state.

With simulated layerwise depolarizing noise `p=0.02`, the 5,000-shot mean energy degraded from about -10.55 to -3.51 and mean feasibility from 0.9661 to 0.8531. This is a noise-model result, not hardware execution.

5.2 VQE, quantum optimizers and QNN

For a two-qubit Hamiltonian with exact ground energy -1.5628499608:

Optimizer	Iterations	Final error	Device executions
Parameter-shift Adam	80	6.024e-4	1,442
SPSA	200	3.225e-4	802
Block-diagonal QNG	40	9.51e-5	922

Parameter shift matched finite differences within `3.62e-11`. SPSA uses two objective evaluations per stochastic gradient regardless of parameter count but is noisy. QNG obtained the smallest final error here while paying for gradient and metric estimation.

A two-qubit QNN trained across three seeds obtained mean train/test accuracy 0.8542/0.9583 and required 4,344 device executions per seed. The eight-example test set is only a functional demonstration. A random-circuit study found global-observable gradient variance decreasing from 0.1703 to 0.006989 between two and six qubits at depth five. This is a small barren-plateau trend, not a general proof.

5.3 Correctness and limitations

The independent verifier passed 34 checks covering basis energies, QAOA bounds, shots, noisy simulation, VQE bounds, derivative accounting, QNG metrics, QNN costs, and gradient variances. All results are local simulations; data loading, calibration, device topology, queueing, and real hardware errors are absent.

Code and evidence: quantum_optimization.py, Track C details, and `results/track_c_quantum/20261028T093727.588182Z/`.

6. Track D - Submodular, parallel, distributed and multi-task optimization

6.1 Submodular maximum coverage

The objective selects at most four sets to maximize union size. Exhaustive checks confirmed normalization, non-negativity, monotonicity, and 5,103 diminishing-return inequalities. Across ten 12-set instances, greedy achieved mean ratio 0.9865 and minimum ratio 0.9375 relative to exhaustive optimum. The `1-1/e` guarantee is claimed only because the objective is normalized, monotone and submodular with a pure cardinality constraint and exact marginal gains.

6.2 Parallel scheduling

Thirty-two unequal CPU tasks were executed serially and with static balance, a centralized dynamic queue, and actual work stealing through per-worker deques. Every method returned identical checksums.

Method	Workers	Mean time	Speedup	Efficiency
Serial	1	0.820 s	1.000	1.000
Static	4	0.524 s	1.564	0.391
Dynamic queue	4	0.524 s	1.565	0.391
Work stealing	4	0.540 s	1.519	0.380

Work stealing preserved more logical locality but was slightly slower because locks and steals cost time. At four workers the empirical Amdahl serial fraction was about 0.52-0.55. A scaled work-stealing experiment measured 1.542x speedup, below the 2.366 Gustafson prediction; spawn, synchronization, and communication violate the simplified constant-serial-fraction assumption.

Small data, model, and pipeline parallel demonstrations were all slower than serial execution. Their speedups were 0.0662, 0.0000559, and 0.0337 because startup and communication dominated computation. This is an important method-selection result: parallel structure alone does not imply speedup.

6.3 FedAvg

Four unequal clients train seven logistic-regression parameters. Weighted FedAvg uses `theta = sum_k (n_k/N)theta_k`. Under the same total compute, increasing local steps from one to ten reduced transmitted bytes from 13,440 to 1,344 without reducing mean accuracy on this simple convex task. IID and deliberately label-skewed non-IID splits were both tested. This result depends on full participation, deterministic full-batch training, a tiny model, and no network latency.

6.4 MGDA and GradNorm

A shared two-head model compared equal weighting, two-task MGDA, and GradNorm across the same data, initializations, and 120 epochs.

Method	Task 1 accuracy	Task 2 accuracy	Interpretation
Equal	0.8587 +/- 0.0023	0.8600 +/- 0.0174	Best balanced result
MGDA	0.8427 +/- 0.0389	0.8280 +/- 0.0223	Correct gradient objective but worse generalization
GradNorm	0.8627 +/- 0.0061	0.8560 +/- 0.0174	Shifted task trade-off

MGDA's constrained coefficient and GradNorm's learned weight gradients were checked on analytical cases. Neither adaptive weighting method universally improved both tasks.

6.5 Correctness and limitations

The parallel verifier passed 31 checks and the distributed/multi-task verifier passed 30. Output hashes, speedup formulas, real steals, communication bytes, FedAvg equivalence, MGDA cases, and GradNorm gradients were recomputed. Distributed learning was simulated locally and does not include network failures, privacy, compression, client dropout, or heterogeneous devices.

Code and evidence: parallel_optimization.py, distributed_multitask.py, parallel report, distributed report, `results/track_d_parallel/20261028T092127.549032Z/`, and `results/track_d_distributed/20261028T092752.487437Z/`.

7. Track E - Main case study: project-workflow scheduling

7.1 Process and baseline evidence

The chosen real-process abstraction is a project workflow containing non-preemptive tasks, release times, precedence constraints, and three identical workers. The resource consumed is elapsed project completion time. A fixed-order ready-list scheduler is the baseline because it is simple and fast but ignores downstream criticality.

Fifteen deterministic synthetic DAGs contain 7, 14, or 24 jobs. The 14-job baseline mean makespan was 34.4, while exact MILP achieved 32.6. The 24-job baseline was 45.6, while critical-path priority achieved 43.0. These gaps demonstrate an optimization opportunity.

7.2 Formulation

Decision variables are integer start times `s_j`, binary worker assignments `a_jw`, pairwise worker-order variables `o_ijw`, and makespan `Cmax`. The objective is:

```
minimize Cmax
```

subject to one worker per job, release times, precedence constraints, makespan bounds, and disjunctive non-overlap on each worker. A valid big-M is the serial horizon:

```
H = max_j release_j + sum_j duration_j
```

The critical-path lower bound combines total work divided by three and the maximum release-plus-downstream-path length.

7.3 Strategy selection

Method	Variables	Guarantee/scale	Reason selected
Fixed ready list	Discrete dispatch choices	Fast, no quality guarantee	Operational baseline
Critical-path list	Discrete choices plus downstream lengths	Fast heuristic	Prioritizes jobs unlocking long paths
Disjunctive MILP	Binary assignment/order and integer time	Exact only if solver proves optimality	Explicit constraints and bounds
Exhaustive oracle	Dispatch/worker choices	Exact only for seven jobs	Independent correctness reference

Gradient methods are unsuitable because the core decisions are discrete. A QUBO encoding would require many time, assignment, and ordering variables plus penalties, with no clear advantage on the available simulator hardware.

7.4 Results

Jobs	Method	Mean makespan	Mean runtime	Change versus baseline
7	Fixed	24.4	0.0297 ms	baseline
7	Critical path	24.8	0.0272 ms	2.50% worse
7	MILP	24.4	3.792 ms	0%; proven optimal
14	Fixed	34.4	0.0600 ms	baseline
14	Critical path	33.0	0.0587 ms	4.12% better
14	MILP	32.6	34.948 ms	5.29% better; proven optimal
24	Fixed	45.6	0.1204 ms	baseline
24	Critical path	43.0	0.1198 ms	5.69% better
24	MILP	43.2	1004.319 ms	5.33% better; time-limit feasible

Every 24-job MILP hit the one-second limit; mean reported MIP gap was 14.87%. These incumbents are not called optimal. The critical-path heuristic produced slightly better mean schedules within the time budget and was about four orders of magnitude faster. It nevertheless regressed on one small instance, so it is not a dominance rule.

The first AI-generated time-indexed MILP could not find a feasible 24-job incumbent under the limit. Replacing it with a compact disjunctive formulation produced valid incumbents and preserved truthful solver status.

7.5 Correctness and limitations

The verifier passed 17 checks over 135 raw records. It independently validates every schedule, enumerates all five small optima, recomputes bounds and gaps, and checks solver status. Limitations are synthetic deterministic jobs, identical workers, no skills or calendars, no communication, and no uncertainty. A deployed model should use historical duration distributions and robust or stochastic scheduling.

Code and evidence: scheduling.py, Track E details, and `results/track_e_scheduling/20261027T135208.655940Z/`.

8. Track F - Compiler optimization

8.1 Compiler and passes

The toy compiler implements lexer, recursive-descent parser, AST, three-address IR, slot bytecode, bytecode VM, and an independent AST interpreter. Four passes are implemented:

1. constant folding;
2. constant propagation;
3. dead-code elimination;
4. common-subexpression elimination.

The compiler decision is pass ordering. A DP chooses a permutation of all four passes under equal pass cost and a budget of four. The objective minimizes deterministic bytecode size on three tuning programs. The selected order was CSE -> propagation -> folding -> DCE, compared with fixed folding -> propagation -> DCE -> CSE.

8.2 Results

Pipeline	Mean instructions	Mean bytecode	Compile time	Execute time
Unoptimized	51.000	1643.89 B	0.3258 ms	0.01301 ms
Fixed order	40.000	1167.33 B	0.3831 ms	0.00898 ms
DP order	40.222	1153.11 B	0.4684 ms	0.00863 ms

The fixed order reduced instructions by 21.57% and bytecode size by 28.99%. The DP order achieved the selected objective and reduced size by 29.85%, but compiled more slowly and produced slightly more mean instructions than the fixed order. The tuning and evaluation programs are hash-disjoint.

8.3 Correctness and limitations

All optimized outputs match the AST interpreter and a separately implemented VM. The verifier passed 17 checks across 270 raw records and reproduced DP selection. The language has no branches, loops, functions, aliasing, exceptions, or fixed-width overflow, so these results do not predict LLVM/GCC behavior.

Code and evidence: toy_compiler.py, Track F details, and `results/track_f_compiler/20261027T140035.600751Z/`.

9. Track G - xv6 operating-system optimization

9.1 Kernel change and objective

Official xv6-riscv is pinned at commit `e93dabeb98431235c6e1a101cb96c73c923fdbc4`. The baseline uses the original round-robin scheduler. The optimized kernel adds weighted-stride scheduling plus `setweight` and `getschedstats` system calls. Each runnable process has weight, stride, virtual pass, dispatch count, and run ticks. The scheduler selects the runnable process with minimum virtual pass.

The objective is to give explicitly high-weight CPU-bound processes a larger share and lower completion time while preserving kernel correctness. One kernel feature benchmarked on three workloads satisfies the assignment minimum; a second feature was not implemented.

9.2 Results

Metric	Round robin	Stride	Outcome
Weight-1 CPU completion	5 ticks	6 ticks	20% worse
Weight-3 CPU completion	3 ticks	3 ticks	tie
Weight-6 CPU completion	4 ticks	3 ticks	25% better
Mixed background completion	2 ticks	2 ticks	tie
Mixed interactive completion	5 ticks	5 ticks	tie
High-weight latency target	2 ticks	3 ticks	50% worse

The result is a trade-off rather than universal improvement. The intended high-weight CPU job finishes sooner, but the low-weight job is delayed. The latency target regresses because it inherits virtual time and can change its own weight only after its first dispatch. A parent-controlled interface or carefully bounded arrival policy could address this but was not implemented.

9.3 Correctness and limitations

Both kernels built and booted. All 48 paired benchmark records have matching checksums. The modified kernel ran all 74 upstream tests in order, printed `ALL TESTS PASSED`, returned to the shell, and produced no panic. The independent verifier passed 22 checks, and the patch applies cleanly to the pinned source.

Measurements used one emulated CPU and coarse approximately 0.1-second ticks. They do not establish multiprocessor scalability, bare-metal performance, long-run proportional share, or production fairness. The optional second kernel feature was not implemented.

Code and evidence: Track G details, xv6 patch, and `results/G_stride/20261027T141300.613368Z/`.

10. Track H - DBMS optimization

10.1 Formulation

SQLite executes a six-query analytical workload over 100,000 orders, 5,000 customers, and 1,000 products. Seven candidate indexes are available. For binary index decisions `x_i`, measured individual benefits `b_i`, measured pages `p_i`, and page budget 896:

```
maximize sum_i b_i x_i
subject to sum_i p_i x_i <= 896
x_i in {0,1}
```

The baseline has no added candidate indexes. Greedy selects by measured benefit per page. Dynamic programming solves the additive knapsack exactly, with exhaustive enumeration as an independent oracle. An all-index reference shows the performance/storage ceiling.

10.2 Results

Design	Added storage	Preparation	Workload latency	Throughput	Improvement
Baseline	0	0.842 ms	17.301 +/- 0.162 ms	346.83 queries/s	baseline

Design	Added storage	Preparation	Workload latency	Throughput	Improvement
Greedy	3.117 MiB	49.796 ms	9.305 +/- 0.039 ms	644.83 queries/s	46.22%
Dynamic programming	3.117 MiB	same design	9.305 +/- 0.039 ms	644.83 queries/s	46.22%
All indexes	7.781 MiB	146.119 ms	5.163 +/- 0.258 ms	1164.41 queries/s	70.16%

Greedy happened to match exact DP for the additive model. The selected indexes fit the 3.5 MiB budget and change several full scans to indexed searches. Query results remain identical through ordered row hashes.

The additive model misses index interactions: product-category and product indexes have zero individual measured benefit but work together in the all-index design. DP is exact for the formulated additive objective, not for true nonlinear workload latency. More indexes also cost storage, build time, and write amplification.

10.3 Correctness and limitations

The verifier passed 20 checks across 168 timing records. It regenerated all 100,000 orders, rebuilt designs, reran all queries, checked ordered result hashes, reproduced index sizes, selections, plans, and the exact additive optimum. The experiment is read-only and does not measure update overhead or concurrent transactions.

Code and evidence: database_optimization.py, Track H details, and `results/track_h_database/20261028T084139.426860Z/`.

11. Track I - Quantum algorithms and QML comparison

11.1 Algorithms

Deutsch-Jozsa, Bernstein-Vazirani, and Grover use directly written circuits and oracles. QFT, QPE, compiled Shor order finding, and a unitary time-dependent Ising annealing model complete the assignment-listed portfolio.

Algorithm	Measured result
Deutsch-Jozsa	Constant and balanced parity oracles classified with one query
Bernstein-Vazirani	Secret `1011` recovered with one query versus four deterministic classical queries
Grover	Marked `101` with 0.9453125 success using two oracle calls
QFT	Analytic-state fidelity 1.0 and inverse round-trip probability 1.0
QPE	Phase 3/8 recovered with probability approximately 1.0
Compiled Shor	Order 4 and factors 3 and 5 for fixed N=15, a=2
Quantum annealing	Ground-state success rose from 0.2890 at T=0.5 to 0.9946 at T=8

For the fixed Grover target, linear search used six queries; its uniform-target expectation is 4.5 and worst case eight. Nevertheless, the quantum simulator averaged 0.001019 seconds versus 2.23e-7 seconds classically, roughly 4,563 times slower. Query complexity and simulator speed are different claims.

The Shor example uses a compiled modular multiplication unitary for one small instance and is not a general factoring implementation. The annealing experiment uses Trotterized closed-system gate-model evolution, not classical simulated annealing and not annealing hardware. Real sparse-connectivity annealers may require physical-qubit chains, chain-strength tuning, and broken-chain decoding.

11.2 QML versus classical learning

All models use the same deterministic 24/8/8 split of a 40-example two-dimensional dataset. Preprocessing statistics are fit only to training data. Validation selects hyperparameters before test evaluation.

Model	Test accuracy	Runtime	Parameters	Circuit executions
VQC, three-seed mean	1.000	6.658 s	6	12,520 mean
Classical MLP, three-seed mean	1.000	0.00859 s	13	0
Quantum kernel	1.000	0.499 s	no trained weights	684
Classical RBF kernel	1.000	0.000701 s	n/a	0

Model	Test accuracy	Runtime	Parameters	Circuit executions
Noisy VQC, p=0.02 and 1,000 shots	1.000	inference only	6	8 per seed

There is no measured QML advantage. The ideal VQC is approximately 775 times slower than the classical MLP while achieving the same accuracy. The test set is very small, and the simulated noise model does not represent all hardware effects. State preparation and general classical-data loading costs are also excluded.

Compared with Track D, parallel execution produced a real measured 1.565x wall-clock speedup on this CPU workload. Grover produced only an oracle-query advantage, and every quantum result was simulated. This distinction prevents an unsupported quantum speedup claim.

11.3 Correctness and limitations

The independent verifier passed 41 checks and freshly reran DJ, BV, Grover, and annealing circuits. It checks probability normalization, exact phases and factors, dataset separation, circuit counts, finite shots, noise labels, kernel positive-semidefiniteness, and simulator disclosure.

Code and evidence: quantum_algorithms.py, Track I details, and `results/track_i_quantum/20261029T180846.017650Z/`.

12. Track J - Software-engineering optimization

12.1 Process and formulation

The realistic codebase is `more-itertools` 10.7.0, pinned by archive hash and distributed under the MIT License. Its production source has 4,728 nonblank, non-comment physical lines and 671 tests.

For test decision `x_i`, measured duration `t_i`, and coverage relationship `a_li`:

```
minimize sum_i t_i x_i
subject to sum_i a_li x_i >= 1 for every target line l
x_i in {0,1}
```

The full profile executes 2,030 production lines, including 1,722 selectable line constraints. The baseline runs all tests. Alternatives are lexical first-cover, greedy new-lines-per-second, and exact ILP set cover.

12.2 Results

Method	Tests	Wall time	Reduction	Coverage retained
Full	671	6.999 +/- 0.009 s	baseline	2,030/2,030
Naive	304	5.682 +/- 0.025 s	18.81%	2,030/2,030
Greedy	263	5.399 +/- 0.005 s	22.86%	2,030/2,030
ILP	242	5.392 +/- 0.015 s	22.96%	2,030/2,030

Complete experimental profiling cost 92.60 seconds. At 1.607 seconds saved per run, that cost amortizes after roughly 58 repeated CI executions if coverage relationships remain stable. Startup, collection, and a few expensive tests explain why a 63.9% test-count reduction yields only a 22.96% wall-time reduction.

Every selected suite independently re-executed all target lines. Two temporary deterministic mutants were killed by all four suites. This small mutation sample does not prove equal fault detection: line coverage can miss value, branch, timing, and interaction faults.

12.3 Correctness and limitations

The verifier passed 22 checks, reproducing selections, source size, license, archive hash, optimal ILP status, target coverage, runtime samples, and mutation results. Profiles must be refreshed when code, tests, Python, or tooling changes. A maintainable CI policy should periodically retain the full suite and use reduced suites only when the risk is acceptable.

Code and evidence: test_selection.py, Track J details, and `results/track_j_test_selection/20261028T090927.339437Z/`.

13. Cross-track method selection

The experiments support the following strategy:

Problem characteristic	Suitable methods demonstrated	Important rejection/limitation
Smooth continuous, small and well-conditioned	GD or Newton	Adaptive overhead may not help
Stochastic/noisy gradients	SGD, mini-batch, momentum, adaptive methods, decay	Requires tuning and repeated seeds
Small binary problem with hard constraints	DP, Branch-and-Bound, ILP	Exactness may become expensive with scale
Fast combinatorial decision	Greedy	Verify feasibility; do not claim optimality without assumptions
Monotone submodular cardinality selection	Greedy with `1-1/e` guarantee	Guarantee does not transfer to unrelated objectives
Discrete scheduling with moderate size	Critical-path heuristic or MILP	MILP time limits can prevent proof or even an incumbent
Parallel workload with enough computation per task	Static/dynamic/work stealing	Startup and communication dominate fine-grained work
Communication-constrained distributed learning	FedAvg with controlled local steps	Non-IID data and system heterogeneity can change results
Variational quantum task	QAOA/VQE/QNN with classical optimizer	Simulation, shots, noise, loading, and trainability costs dominate small examples
Budgeted physical database design	Greedy or exact knapsack/ILP/DP	Additive benefits miss index interactions
Repeated CI selection	Greedy/ILP set cover	Coverage does not guarantee equal fault detection

14. AI-assisted development and critical review

Codex was used for planning, code generation, debugging, tests, verification, and documentation across Tracks A-J. The exact model identifier was unavailable and is not invented. Per-track prompt records are stored under `prompts/`, and the tool register is in AI_TOOLS.md.

Eleven genuine AI-generated issues are documented in AI_FAILURES.md. Examples include an xv6 API mismatch, console framing races, a no-incumbent MILP, unstable pytest root discovery, timing-dependent coverage, an interactive plotting backend, unsupported multiprocessing IPC, and queued quantum Hamiltonian arithmetic that produced non-normalized states. These failures demonstrate why generated code required compilation, invariants, independent implementations, and full regression tests.

The assignment's required comparison with a second genuine AI tool is **not complete**. AI_TOOL_COMPARISON.md truthfully records that only Codex evidence is available. A second tool's output, model/version, original response/export, prompt iterations, correctness, and observed timing must be added by the student; this report does not fabricate them. The first complete verified repository snapshot is commit `9757572`; historical per-step commit links do not exist because Git was initialized only at final handoff.

15. Answers to the ten final questions

1. What process was chosen, and what evidence showed that it needed optimization?

The main process was project-workflow scheduling: assigning non-preemptive jobs with release times and precedence constraints to three identical workers while minimizing makespan. The fixed-order baseline averaged makespan 34.4 on 14-job instances versus 32.6 for exact MILP, and 45.6 on 24-job instances versus 43.0 for critical-path priority. The reproducible gaps show idle time and poor task ordering that optimization can reduce.

2. How were the objective function and constraints formulated?

The main decision variables are integer job start times, binary worker assignments, and binary pair-order choices. The objective minimizes `Cmax`. Constraints enforce release times, one worker per job, precedence, non-overlap on each worker, and completion before `Cmax`. A serial horizon provides a valid big-M. Other tracks similarly formulate ridge loss, binary knapsack, QUBO penalties, submodular coverage, index budgets, and test-set cover.

3. Which Chapter 6 methods were tested, and why were they appropriate?

The portfolio includes GD, SGD, mini-batch SGD, Momentum, NAG, AdaGrad, RMSProp, Adam, Newton, Gauss-Newton, backpropagation, Fisher/diagonal/low-rank curvature and HVPs; greedy, DP, ILP, Branch-and-Bound, QUBO and Ising; QAOA, VQE, PQCs/QNNs, parameter shift, SPSA and quantum natural gradient; submodular greedy; static/dynamic/work-stealing parallelism; data/model/pipeline parallelism; FedAvg, MGDA and GradNorm; Deutsch-Jozsa, Bernstein-Vazirani, Grover, QFT, QPE, compiled Shor, quantum annealing, variational classifiers and quantum kernels. Methods were assigned to benchmarks matching their variable types and assumptions rather than forced into the main case study.

4. How did optimized solutions compare with baselines?

Improvements depended on the process. The 24-job scheduling heuristic reduced makespan 5.69%; compiler passes reduced bytecode size 29.85%; database indexes reduced latency 46.22%; and ILP test selection reduced CI wall time 22.96%. Parallel scheduling reached 1.565x speedup. Some methods produced no improvement: quantum ML matched classical accuracy at far higher cost, fine-grained process parallelism was slower than serial, and xv6 stride scheduling improved one high-weight CPU job while regressing the latency workload.

5. What trade-offs, failures, or limitations were discovered?

Exact methods trade runtime for proof; heuristics trade guarantees for speed. More indexes trade query latency for storage/build/write cost. Parallelism trades computation for startup, synchronization, communication, and locality overhead. Quantum methods incur simulation, finite-shot, noise, circuit-evaluation, and data-loading costs. Coverage-based test reduction can reduce fault-detection redundancy. Eleven AI-generated failures were found and corrected. Synthetic data, small problem sizes, simulated quantum hardware, and one-CPU QEMU limit external validity.

6. What evidence demonstrates correctness and reproducibility?

All 68 unit/integration tests pass. Every selected track verifier passed during the final audit. Evidence includes exhaustive knapsack and small-scheduling oracles, all-state QUBO/Ising checks, analytical gradients and Hessians, independent VMs, output hashes, ordered SQL result hashes, full dataset regeneration, quantum probability normalization, mutation tests, xv6 patch dry-run, and all 74 upstream xv6 usertests. Configurations, raw records, seeds, timings, plots, solver status, and environment versions are retained under `results/` and `configs/`.

7. How could the approach adapt if scale, data, constraints, or hardware changed?

Larger scheduling instances would use decomposition, stronger MILP cuts, CP-SAT, rolling horizons, or learned/robust duration models. Large knapsack capacities may favor MILP or approximation methods over DP. Distributed learning would need partial participation, compression, privacy, asynchronous updates, and network measurement. Database design should model index interactions and writes. Parallel work needs persistent pools and higher computation-to-communication ratios. Quantum experiments on hardware would require transpilation, calibration-aware layouts, error mitigation, shot budgeting, and explicit queue costs.

8. For compiler, xv6 and DBMS tracks, what was implemented and how was correctness preserved?

The compiler implements four passes and DP pass ordering; optimized bytecode is compared with an AST interpreter and independent VM. xv6 implements weighted-stride scheduling and two statistics/control system calls; both kernels run identical workloads, paired checksums match, the patch applies to the pinned source, and the modified kernel passes all 74 upstream tests. The DBMS track optimizes seven candidate indexes under a storage budget using greedy and exact DP; query plans, storage, latency and throughput are measured, while ordered query-result hashes remain identical.

9. What advantages and limits did quantum and parallel experiments reveal?

Grover reduced oracle queries and variational algorithms successfully optimized tiny quantum objectives, but local simulation was slower than classical computation and QML showed no accuracy advantage. Noise and finite shots degraded or destabilized estimates, while data loading and hardware embedding remain unmeasured costs. Parallel scheduling produced a real 1.565x speedup for coarse CPU tasks, but data/model/pipeline demonstrations were slower when communication dominated. Quantum advantage here is theoretical/query-based; parallel advantage was directly measured in wall time.

10. Which AI tools were used, how did their outputs differ, and which errors were fixed?

Codex was the only AI tool with preserved project evidence. It supported planning, implementation, debugging, tests, and documentation. Eleven genuine errors were fixed, including invalid APIs, brittle output parsing, an ineffective MILP formulation, nondeterministic coverage assumptions, unsuitable IPC, and invalid quantum operator queuing. A required empirical comparison with a second AI tool has not yet been performed, so no difference in correctness, hallucination rate, speed, or prompt iterations is claimed. This is a known submission gap, not concealed evidence.

16. Reproducibility map

Track	Runner	Independent verifier	Selected result
A	`scripts/run_track_a.py`	`scripts/verify_track_a.py`	`results/track_a_continuous/20261027T134547.158590Z/`
B	`scripts/run_track_b.py`	`scripts/verify_track_b.py`	`results/track_b_discrete/20261027T133954.808190Z/`
C	`scripts/run_track_c.py`	`scripts/verify_track_c.py`	`results/track_c_quantum/20261028T093727.588182Z/`
D parallel	`scripts/run_track_d_parallel.py`	`scripts/verify_track_d_parallel.py`	`results/track_d_parallel/20261028T092127.549032Z/`
D distributed	`scripts/run_track_d_distributed.py`	`scripts/verify_track_d_distributed.py`	`results/track_d_distributed/20261028T092752.487437Z/`
E	`scripts/run_track_e.py`	`scripts/verify_track_e.py`	`results/track_e_scheduling/20261027T135208.655940Z/`
F	`scripts/run_track_f.py`	`scripts/verify_track_f.py`	`results/track_f_compiler/20261027T140035.600751Z/`
G	`scripts/run_track_g.py`	`scripts/verify_track_g.py`	`results/G_stride/20261027T141300.613368Z/`
H	`scripts/run_track_h.py`	`scripts/verify_track_h.py`	`results/track_h_database/20261028T084139.426860Z/`
I	`scripts/run_track_i.py`	`scripts/verify_track_i.py`	`results/track_i_quantum/20261029T180846.017650Z/`
J	`scripts/run_track_j.py`	`scripts/verify_track_j.py`	`results/track_j_test_selection/20261028T090927.339437Z/`

17. Conclusion

The project demonstrates that optimization is not synonymous with choosing the most sophisticated algorithm. Improvements arise when the formulation matches the process, constraints are enforced correctly, overhead is measured, and claims are limited to evidence. Fast heuristics were often the most useful operational choice; exact methods supplied ground truth and bounds at higher cost; parallelism helped only when work outweighed coordination; and the quantum simulations demonstrated algorithms without demonstrating practical advantage.

The experimental A-J portfolio and report are complete at the assignment's minimum track scope. The work is not yet fully submission-complete because the exact taught Chapter 6 list has not been supplied, the mandatory genuine second-AI-tool comparison is missing, commit traceability/public GitHub publication remain student-managed, and full chat exports/screenshots were not captured. These limitations must remain visible in the final submission rather than being silently marked complete.